Clase 40 — Corrección de errores y ARQ

Bloque: E (Capa de enlace) · **Semana:** 14 · **Duración estimada:** 115 min **Correlativa:** Clase 39 (detección de errores: paridad, checksum y CRC) · **Sigue con:** Clase 41 (acceso al medio: ALOHA, CSMA/CD y CSMA/CA) **Material de referencia:** Tanenbaum cap. 3 §3.3–3.5 y cap. 4 · Kurose §5.3 · RFC 2018 (SACK)

🎯 Al terminar esta clase vas a poder

1. Explicar por qué **detectar** y **corregir** son problemas distintos, y en qué medios conviene **corregir en el receptor** en vez de **retransmitir**.
2. Definir la **distancia de Hamming** y derivar las capacidades de **SEC** y **SECDED**.
3. **Derivar los bits de paridad** de un código de Hamming de `m` bits de datos y `r` de paridad, y explicar de dónde sale la condición `2^r ≥ m + r + 1`.
4. Ejecutar el **procedimiento de corrección completo** (cálculo del **síndrome** y ubicación del bit erróneo) sobre un ejemplo numérico.
5. Calcular la **eficiencia de Stop-and-Wait** con la fórmula `1/(1+2a)` y explicar por qué depende del RTT.
6. Describir **Go-Back-N** y **Selective Repeat** con sus diagramas temporales, y decir **por qué TCP no es exactamente ninguno de los dos** salvo con SACK.
7. Justificar **por qué Selective Repeat exige un buffer en el receptor** y ACK no acumulados.
8. Resolver completo el **Examen 2009-03 Pregunta 1** (10 puntos).

> **Prerequisito del examen:** `2009-03 P1` (10 pts — el concepto y el funcionamiento de Go-Back-N), `2007-03 Problema 1` (30 pts — protocolo de capa 2 con ARQ de tipo **Selective Repeat** y *piggybacking*), `2007-07 P2` (5 pts — mecanismos de detección y corrección de errores de la capa de enlace), `2010-07 Problema 1` (30 pts — protocolo *stop-and-wait* semifiable para video), `2011-03 P4` (10 pts — el diagrama TCP con y sin pérdida del primer segmento, que es Go-Back-N puro).
>
> ⚠️ Esta clase tiene **cálculo de verdad**. El código de Hamming se hace bit por bit y los tres ARQ se dibujan con líneas de tiempo. Hacé los ejercicios con lápiz y papel antes de mirar la resolución.

PARTE 1 — TEORÍA

1.1 Detectar no es corregir: por qué la pregunta importa

En la clase anterior (Clase 39) vimos los **detores**: paridad, paridad 2D, checksum y CRC. Todos hacen **lo mismo**: un receptor devuelve "OK" o "error". Cuando devuelve "error", el receptor **no sabe cuál bit** está mal, y por lo tanto **no puede arreglarlo**: lo único que puede hacer es **tirar el paquete y pedir que lo manden de nuevo**.

```
DETECTAR                          CORREGIR
────────                          ────────
"¿hay error?"  sí/no              "¿cuál bit está mal?"  → lo arreglo
coste: 1..32 bits de control      coste: r bits de paridad + lógica
si hay error: OBLIGATORIO          si hay error: se repara solo, sin
retransmitir                      tocar la red
```

La razón económica es el RTT. Corregir en el receptor es gratis en términos de red: no se transmite nada. Retransmitir, en cambio, cuesta:

```
EMISOR ──(trama)──► R1 ──► ... ──► R4 ──(trama)──► RECEPTOR
       ◄──────────── ACK ◄──────────────────────────────────

Si la trama tenía error y se retransmite:
  EMISOR ──(trama)──► ... ──► RECEPTOR          t_frame
         ◄──────────────── ACK ─────────        t_prop
         ◄──────────────── ACK ─────────        t_prop   (el ACK vuelve)
         ──(trama)──────► ... ──► RECEPTOR     t_frame
         ◄──────────────── ACK ─────────        t_prop
         ◄──────────────── ACK ─────────        t_prop

COSTE DE RETRANSMITIR = 2·t_frame + 2·t_prop  = un RTT entero
```

⚠️ **Cuando conviene corregir y no retransmitir:** en los medios donde el **RTT es enorme comparado con el tiempo de transmisión**:

Medio	`t_prop` (ida)	`t_frame` (1 KB)	`a = t_prop/t_frame`	`U` de Stop-and-Wait
LAN a 10 Mb/s (100 m)	0,5 µs	800 µs	0,0006	**99,9 %**
Enlace dedicado a 2 Mb/s (100 km)	500 µs	4 ms	0,125	80,0 %
Fibra dedicada (1000 km, 2 Gb/s)	5 ms	4 µs	**1 250**	**0,04 %**
Enlace satelital GEO (36 000 km)	120 ms	0,8 ms	**150**	**0,33 %**
Radio de larga distancia	100 ms+	—	enorme	casi 0

⚠️ **Atención con la fila del satélite:** es 0,33 %, no 3,3 %. Recalculo: `1/(1+2·150) = 1/301 = 0,00332 = 0,33 %`. **Una mainframe de 2011 sobre un enlace satelital GEO tenía menos del 1 % de Utilización**

del enlace. Por eso en los enlaces demuy largo RTT tiene sentido **corregir** (FEC, *forward error correction*) y no retransmitir: el costo de una corrección en el receptor es cero en la red.

⚠️ **El otro extremo de la balanza:** en un medio **muy ruidoso**, si la tasa de error de bits es alta, retransmitir **todo** es inútil: se pierde el tiempo y se vuelve a perder. Ahí también conviene **corregir** (o cambiar el código), porque **ARQ con tasa de error alta se vuelve un problema**. Ésa es la razón por la que el Wi-Fi y el DVB usan **FEC** y **no** ARQ.

1.2 La distancia de Hamming y los dos códigos que hay que saber

La definición

```
La DISTANCIA DE HAMMING entre dos palabras de la misma longitud
d(A,B) es la CANTIDAD de posiciones en las que difieren.

  A = 1 0 1 0 1 0 1 0
  B = 1 0 1 1 1 0 1 1
      ─   ─     ─     ─
  d(A,B) = 4

Equivalentemente:  d(A,B) = peso(A XOR B)
```

⚠️ **Por qué la distancia de Hamming es LA medida:** porque un error de `k` bits **siempre** transforma una palabra en otra que está a distancia `≤ k`. Si el mínimo `d` de un código es `d_min`, entonces:

```
d errores realmente pueden mover una palabra            → distancia d
d_min − 1 errores NUNCA mueven una palabra fuera del      → undetectable
                                                          código
```

De ahí salen las dos capacidades, y son las que hay que memorizar:

	Detecta	Corrige	`d_min` mínimo
SEC (*Single Error Correcting*)	`d ≤ 1`	**1** bit	**2**
SECDED (*Single Error Correcting, Double Error Detecting*)	`d ≤ 2`	**1** bit	**3**
(doble error, no corrige 2)			
Código con `d_min = 4`	`d ≤ 3`	**1** bit (+ sabe que hay 2)	**4**

⚠️ **El error conceptual más caro de esta clase: la paridad detecta 1 error y corrige 0**. **Hamming SEC detecta 1 y corrige 1**. **Hamming SECDED detecta 2 y corrige 1.** Se preguntan mucho, y se confunden porque las tres cosas suenan parecido.

⚠️ **¿Por qué SECDED necesita `d_min = 3` y no 2?** Porque con `d_min = 2` no se puede distinguir "1 error" de "2 errores": las dos cosas dan distancia 2, y el receptor solo tiene una palabra de referencia. Con `d_min = 3`, un error de 1 bit da distancia 1 (distinguible), uno de 2 bits da distancia 2, y uno de 3 da distancia 3. Como el receptor **solo sabe corregir de 0**, descarta todo lo que sea ≥ 2. Por eso SECDED **detecta 2** aunque solo **corrija 1**.

1.3 Los bits de paridad: la tabla de potencias de 2 y la condición `2^r ≥ m + r + 1`

La intuición: cada bit de paridad "vigila" un subconjunto

La idea es que **cada bit de paridad vigila un conjunto distinto de posiciones**, y que hay `r` bits de paridad. Hay que poder distinguir `r` posiciones más "nada":

```
Hay r+1Casos que el receptor tiene que diferenciar:
  · "no hay error en ninguno"                          → el código está bien
  · "el bit de paridad 1 tiene error"
  · "el bit de paridad 2 tiene error"
  · ...
  · "el bit de paridad r tiene error"
  · "el bit de datos j tiene error"   (¡j puede ser CUALQUIERA de m!)

En total:  r (paridades) + m (datos) + 1 (sin error)
                                                =  m + r + 1  casos

Cada caso se codifica con un patrón distinto de r bits ⇒
     2^r  ≥  m + r + 1
```

La tabla que hay que saber de memoria (con `m` datos y `r` paridades):

`m` (datos)	`r` (paridad)	`m + r + 1`	`2^r`	¿Alcanza?	Total de bits
1	2	4	4	sí	3
2	2	5	4	**no**	—
3	2	6	4	**no**	—
4	**3**	**8**	**8**	**sí**	**7**
5	3	9	8	**no**	—
6	3	10	8	**no**	—
7	3	11	8	**no**	—
8	4	13	16	sí	12
16	5	22	32	sí	21
32	6	39	64	sí	38
64	7	72	128	sí	71

⚠️ **El resultado clave que hay que entender: el SEC no funciona para 5, 6 ni 7 bits de datos.** Es contraintuitivo: uno esperaría que con más datos it'd mieux. La razón es que **cada bit de dato nuevo necesita su propio patrón de síndrome** y los patrones disponibles son `2^r`, que crecen **más lento** que `m + r + 1`. Para esos casos hay que usar **códigos cíclicos** (BCH, Reed-Solomon) o **SEC-DED con bit de paridad global**.

⚠️ **Un detalle de la condición que cambia según el texto:** muchos textos usan la forma **ligeramente más simple** `2^r ≥ m + 1` (ignorando los `r` bits de paridad entre los casos a distinguir). Con `m = 4` da

`2^3 = 8 ≥ 5`, y el resultado es el mismo (`r = 3`). La forma con `+ r + 1` es la rigurosa. **En un examen, usá la que te parezca que pide el enunciado y mencioná la otra.**

Dónde se colocan los bits de paridad

Los bits de paridad van en las posiciones que son **potencias de 2**: 1, 2, 4, 8, 16, ...

```
┌──────────────────────────────────────────────────────────┐
│  palabra completa de 7 bits (m=4, r=3):                  │
│                                                          │
│  posición   1   2   3   4   5   6   7                    │
│           ┌───┬───┬───┬───┬───┬───┬───┐                  │
│  contenido │P1 │P2 │D1 │P3 │D2 │D3 │D4 │                │
│           └───┴───┴───┴───┴───┴───┴───┘                  │
│   ↑    ↑           ↑       ↑   ↑   ↑   ↑                 │
│  2^0  2^1         2^2     (datos, sin paridad)           │
│                                                          │
│  P1 vigila las posiciones cuyo binario tiene el bit 0 = 1:  │
│      1 (001), 3 (011), 5 (101), 7 (111)                  │
│  P2 vigila las posiciones cuyo binario tiene el bit 1 = 1:  │
│      2 (010), 3 (011), 6 (110), 7 (111)                  │
│  P3 (pos 4) vigila las que tienen el bit 2 = 1:          │
│      4 (100), 5 (101), 6 (110), 7 (111)                  │
└──────────────────────────────────────────────────────────┘
```

⚠️ **El patrón no es arbitrario:** "el bit de paridad de la posición `2^j` vigila a todas las posiciones cuyo número tiene el bit `j` en 1". Es lo que hace que el **síndrome sea directamente el número de la posición del error** (§1.5). Si el patrón fuera otro, el síndrome habría que "traducirlo" y no daría el número.

1.4 Ejemplo completo: derivar los bits de paridad con `m = 4`, `r = 3`

Datos. Quiero enviar `D = 1 0 1 0` (D1=1, D2=0, D3=1, D4=0), con **paridad par** (un grupo es válido si tiene un número **par** de unos).

Paso 1 — Armo la plantilla de 7 posiciones.

```
posición:   1    2    3    4    5    6    7
          P1   P2   D1   P3   D2   D3   D4
valor:     ?    ?    1    ?    0    1    0
```

Paso 2 — Calculo P1. P1 vigila posiciones **1, 3, 5, 7**. De las que ya conozco: D1 (pos 3) = 1, D2 (pos 5) = 0, D4 (pos 7) = 0.

```
grupo P1 = { P1, D1, D2, D4 } = { ?, 1, 0, 0 }
unos conocidos = 1  (solo D1)
Para que el total sea PAR, P1 debe hacer el total PAR:
     1 + P1  tiene que ser par  ⇒  P1 = 1     (1+1 = 2, par ✓)
VERIFICACIÓN: {1, 1, 0, 0} → 2 unos → par ✓
```

Paso 3 — Calculo P2. P2 vigila posiciones **2, 3, 6, 7**: P2, D1 (pos 3) = 1, D3 (pos 6) = 1, D4 (pos 7) = 0.

```
grupo P2 = { P2, D1, D3, D4 } = { ?, 1, 1, 0 }
unos conocidos = 2
Para que sea PAR: 2 + P2 par  ⇒  P2 = 0      (2+0 = 2, par ✓)
VERIFICACIÓN: {0, 1, 1, 0} → 2 unos → par ✓
```

Paso 4 — Calculo P3. P3 (posición 4) vigila posiciones **4, 5, 6, 7**: P3, D2 (pos 5) = 0, D3 (pos 6) = 1, D4 (pos 7) = 0.

```
grupo P3 = { P3, D2, D3, D4 } = { ?, 0, 1, 0 }
unos conocidos = 1
Para que sea PAR: 1 + P3 par  ⇒  P3 = 1       (1+1 = 2, par ✓)
VERIFICACIÓN: {1, 0, 1, 0} → 2 unos → par ✓
```

Paso 5 — La palabra de código completa.

pos 1	pos 2	pos 3	pos 4	pos 5	pos 6	pos 7
P1	P2	D1	P3	D2	D3	D4
1	0	1	1	0	1	0

```
PALABRA DE CÓDIGO =  1 0 1 1 0 1 0  =  "1011010"
```

⚠️ **El "truco" que hay que aplicar siempre:** la paridad de cada grupo es **(número de unos conocidos) mod 2**, y el valor de la paridad es **ese mismo número** (0 si es par, 1 si es impar). Es decir, con paridad par, `P = n_unos_conocidos mod 2`. Con **paridad impar**, `P = (n_unos_conocidos + 1) mod 2`.

⚠️ **El dato de la cátedra que hay que tener:** el **práctico 5 de 2008, ejercicio 4**, es el del **CRC** (`D = 10101010` con el generador de 4 bits de la Figura 5.8) y **ya está resuelto en la Clase 39, §2.4**. Lo que la Figura 5.4 del libro representa es exactamente la plantilla de Hamming de arriba, y este ejercicio es su desarrollo numérico.

1.5 El procedimiento de corrección: el SÍNDROME

Acá está el beautiful del código: **no hay que "buscar" el error**. El receptor recalcula las paridades, y **el número que forman los resultados en binario ES la posición del bit equivocado**.

```
El receptor recibe la palabra R y hace TRES cosas:

1) Recalcula la paridad de cada grupo, SIN tocar la palabra.
     Para el grupo de P1: ¿la paridad que dice el bit 1 de R
     coincide con la que recalculo yo?  Sí/No.
     Lo mismo con P2 y P3.

2) Escribe esos tres Sí/No como un NÚMERO BINARIO de 3 bits:
     (P3, P2, P1)  =  b3 b2 b1

3) Ese número es la POSICIÓN del bit que está mal.
     Si es 0 → no hay error.
     Si es 1..7 → corregir ese bit (voltearlo).
```

⚠️ **Por qué funciona:** el patrón de cada paridad se diseñó justamente para que el conjunto de "qué paridades fallan" identifique **unívocamente** a la posición. Un solo error en la posición `j` **falla exactamente** en las paridades `P_k` donde el binario de `j` tiene el bit `k` en 1. Y como los `r` bits de paridad están en las posiciones 1, 2, 4, 8… (que son exactamente las potencias de 2), el número formado por "qué paridades fallaron" **es** el número de la posición. Es una traducción perfecta.

Ejemplo completo de corrección

Datos. Se envió `1011010`. El canal **invierte el bit de la posición 6** (el `D3`, que era 1 y llega 0).

Lo que llega:

```
┌─────────┬─────────┬─────────┬─────────┬─────────┬─────────┬─────────┐
│ pos 1   │ pos 2   │ pos 3   │ pos 4   │ pos 5   │ pos 6   │ pos 7   │
├─────────┼─────────┼─────────┼─────────┼─────────┼─────────┼─────────┤
│    1    │    0    │    1    │    1    │    0    │  0 ✗    │    0    │
└─────────┴─────────┴─────────┴─────────┴─────────┴─────────┴─────────┘

RECIBIDO = 1 0 1 1 0 0 0  =  "1011000"
```

Paso 1 — Recalculo P1. Grupo P1 = posiciones {1, 3, 5, 7} = {1, 1, 0, 0}.

```
Paridad que DICE el bit 1 recibido:  1
Paridad que CALCULO sobre el grupo:   1 + 1 + 0 + 0 = 2  → par → 0
¿Coinciden?   1 ≠ 0   ⇒  P1 FALLA   (s₁ = 0)
```

Paso 2 — Recalculo P2. Grupo P2 = posiciones {2, 3, 6, 7} = {0, 1, **0**, 0}.

```
Paridad que DICE el bit 2 recibido:  0
Paridad que CALCULO: 0 + 1 + 0 + 0 = 1  → impar → 1
¿Coinciden?   0 ≠ 1   ⇒  P2 FALLA   (s₂ = 1)
```

Paso 3 — Recalculo P3. Grupo P3 = posiciones {4, 5, 6, 7} = {1, 0, **0**, 0}.

```
Paridad que DICE el bit 4 recibido:  1
Paridad que CALCULO: 1 + 0 + 0 + 0 = 1  → impar → 1
¿Coinciden?   1 = 1   ⇒  P3 NO FALLA   (s₃ = 0)
```

Paso 4 — El síndrome.

```
s₃ s₂ s₁  =  0  1  0  =  0b010  =  6
                              ↑
                        "el bit 6 está mal"
```

Paso 5 — Corrijo. Volteo la posición 6 de 0 a 1.

```
┌─────────┬─────────┬─────────┬─────────┬─────────┬─────────┬─────────┐
│    1    │    0    │    1    │    1    │    0    │  1 ←    │    0    │
└─────────┴─────────┴─────────┴─────────┴─────────┴─────────┴─────────┘

CORREGIDO = 1 0 1 1 0 1 0  =  "1011010"  ✓  ¡coincide con lo enviado!
DATOS EXTRAÍDOS = D1 D2 D3 D4 = 1 0 1 0   ✓
```

El caso "no hay error": si la palabra llega bien, los tres chequeos dan "coincide", el síndrome es `000` = 0, y **no se corrige nada**. Por eso el 0 se puede usar como código de "todo bien".

⚠️ **La tabla de síndromes de este ejemplo**, que conviene tener a mano para verificar cualquier otro:

Syndrome ($s_3 s_2 s_1$)	Decimal	Posición	Qué es	¿Debe ser paridad?
000	0	—	sin error	—
001	1	1	**P1**	sí
010	2	2	**P2**	sí
011	3	3	**D1**	no
100	4	4	**P3**	sí
101	5	5	**D2**	no
110	6	6	**D3**	no
111	7	7	**D4**	no

⚠️ **El argumento que demuestra que SEC detecta 1 error:** los 8 síndromes posibles son `0` (sin error) o el número de una de las 7 posiciones. Un solo bit equivocado produce **exactamente un** código distinto del 0, y ese código **no puede** coincidir con el de otra posición porque la tabla es una **biyección**. Por eso SEC detecta y corrige **exactamente 1** y **solo 1**. Con **dos** errores, los síndromes se pueden sumar y dar un valor de una posición válida: **SEC no detecta 2**. Ése es el motivo de que el SECDED agrega un bit de paridad **global** (el bit 9 en posiciones de 1 a 8): un segundo error **siempre** cae en un grupo o en el global de forma que la paridad global queda **impar**, y eso es una señal inequívoca de "hay dos errores, no corrijas".

1.6 ARQ: reintentar en vez de corregir

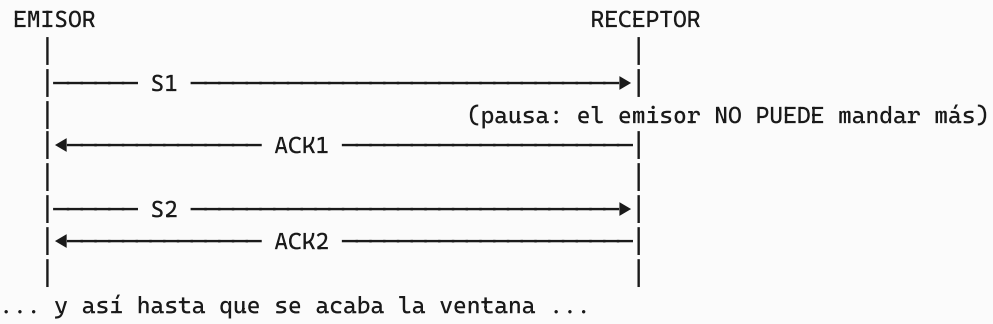
ARQ = *Automatic Repeat reQuest*. Es la estrategia complementaria: **no corrijo, reintento**. Y hay **tres** implementaciones clásicas, que se distinguen por **una sola pregunta**: *cuando se pierde el segmento* k*, ¿qué retransmito?*

```
Stop-and-Wait:     retransmito  SÍ O SÍ  el segmento k (solo ese)
Go-Back-N:         retransmito        k, k+1, k+2, ... (en la ventana)
Selective Repeat:  retransmito  SÍ     k (y solo k)
```

⚠️ Y el **ACK acumulado** es la otra mitad de la historia:

```
ACKs ACUMULADOS:   ACK(k) significa "recibí TODO hasta el k, incluido"
                   ⇒ es 1 ACK para muchos segmentos (eficiente)
ACKs NO ACUMULADOS: ACK(k) significa "recibí el k"
                   ⇒ hace falta 1 ACK por segmento
```

1.6.1 Stop-and-Wait



El problema: el emisor queda **ocioso** `2·t_prop` entre trama y trama. En un enlace de 1 Gb/s de 1000 km con tramas de 1 KB, eso es un desperdicio del **99,9 %**.

La eficiencia (utilización) del enlace:

```
Definimos:   a = t_prop / t_frame
             (el cociente entre el tiempo de propagación y el de
              transmisión de UNA trama)

Ciclo completo de Stop-and-Wait:
     t_frame          (mando la trama)
   + t_prop           (llega)
   + t_prop           (vuelve el ACK, instantáneo)
   + t_frame          (mando la trama siguiente)
   =  2·t_frame + 2·t_prop = 2·t_frame·(1 + a)

Tiempo útil = t_frame

═══════════════════════════════════════════════════════
     U = 1 / (1 + 2a)
═══════════════════════════════════════════════════════
```

La cuenta con números. Trama de **1000 bytes** (8000 bits) sobre un enlace de **1 Mbps**, a **2000 km** de distancia con `v = 2 × 10⁸ m/s`:

```
t_frame = L/R = 8000 bits / 10⁶ bit/s = 8 ms
t_prop  = d/v = 2 000 000 m / 2 × 10⁸ m/s = 10 ms

a = t_prop / t_frame = 10 / 8 = 1,25

U = 1 / (1 + 2a) = 1 / (1 + 2 × 1,25) = 1 / 3,5 = 0,2857 = 28,6 %
```

Otro caso, extremo: enlace **satélital GEO** (36 000 km, `v = 3 × 10⁸ m/s`), trama de 1000 bytes a **10 Mbps**:

```
t_frame = 8000 / 10⁷ = 0,8 ms
t_prop  = 36 000 000 / 3 × 10⁸ = 120 ms

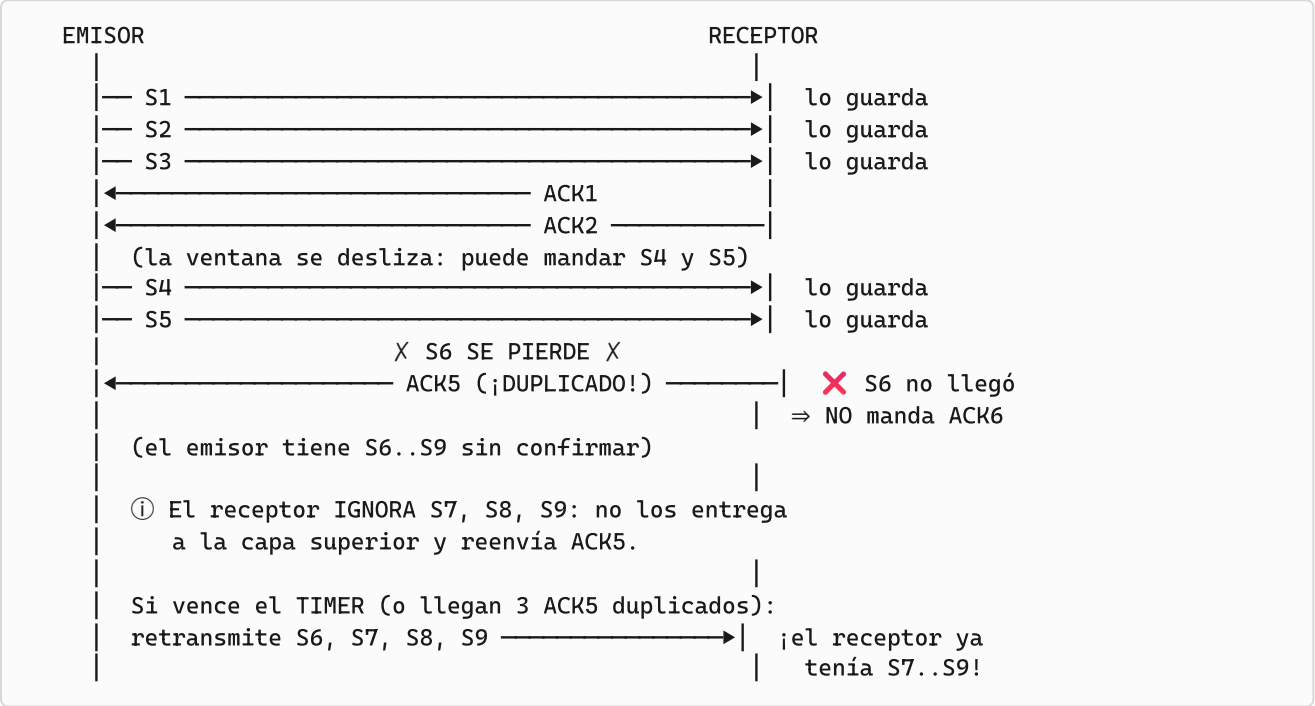
a = 120 / 0,8 = 150

U = 1 / (1 + 300) = 1/301 = 0,00332 = 0,33 %
```

⚠️ **La conclusión para el examen:** Stop-and-Wait es **aceptable** cuando `a` es chico (redes LAN, enlaces locales) y **inservible** cuando `a > 1` (el enlace queda más tiempo ocioso que trabajando). **La respuesta completa es "hay que usar ventanas"**, y de ahí salen los otros dos protocolos.

1.6.2 Go-Back-N

La idea: el emisor puede tener **hasta `N` segmentos sin confirmar** en el medio. El receptor **solo manda ACK acumulados** y, si le llega un segmento **con un hueco**, **lo descarta y reenvía el ACK del que esperaba**.



Las cinco reglas de Go-Back-N (que son los "titulares" de la solución oficial):

```
1. El emisor puede tener hasta N paquetes sin ACK en el pipeline.
2. El receptor SOLO envía ACKs acumulados.
3. El receptor NO envía el ACK de un paquete si hay un hueco.
4. El emisor tiene UN SOLO timer, para el paquete más viejo
   todavía no confirmado.
5. Si el timer vence, retransmite TODOS los paquetes no confirmados
   (desde el más viejo, todos los de la ventana).
```

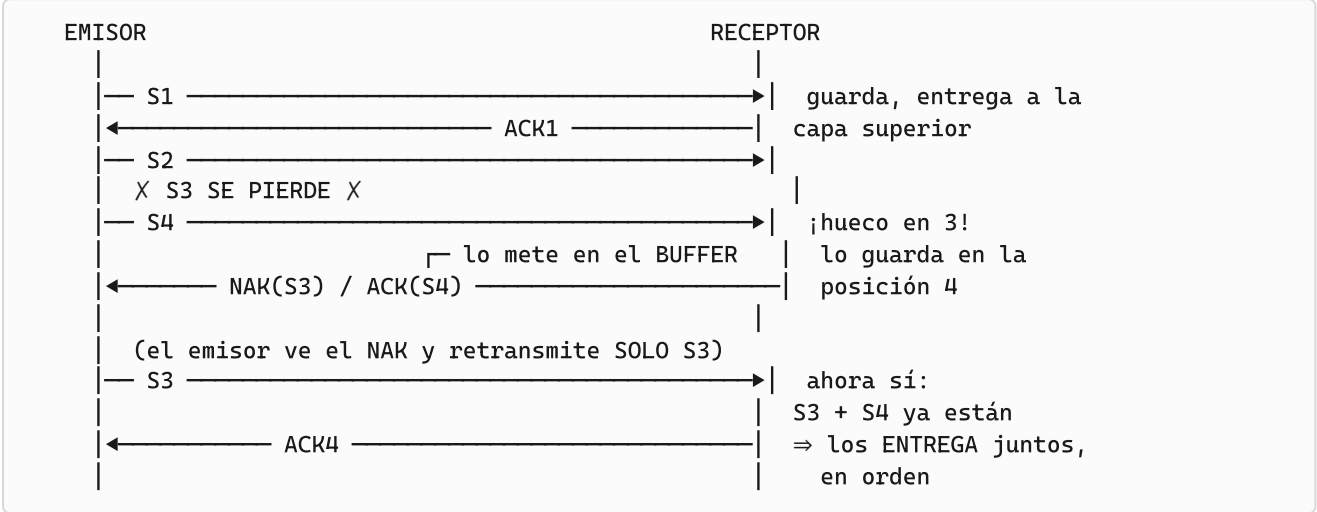
⚠️ **La característica definitoria:** el receptor **descarta** lo que llega después del hueco y **no lo reenvía**. Cuando por fin llega el perdido, el receptor tiene que recibir **de nuevo** todo lo que ya había. Eso se llama **waste** (desperdicio), y es **el precio de la simplicidad**.

⚠️ **La ventaja:** el receptor **no necesita buffer** (o necesita uno de tamaño 1), no necesita **reordenar**, y el ACK acumulado es barato: **un ACK por segmento, no uno por segmento recibido**.

⚠️ **La desventaja: ineficiente** cuando hay pérdida. Una sola pérdida retransmite `N` segmentos. Con `N = 128` (el máximo de TCP), una pérdida retransmite hasta 128 segmentos.

1.6.3 Selective Repeat

La idea: el receptor tiene un **buffer** y **reordena**. Cuando le llega algo fuera de orden, **lo guarda** y **manda un NAK** (o un ACK selectivo) diciendo qué le falta. El emisor retransmite **solo** lo que falta.



Las cuatro condiciones que hacen que funcione:

Condición	Por qué
Buffer de recepción en el receptor (tamaño ≥ `N`)	tiene que guardar los segmentos que llegan fuera de orden
Reordenar al recibir el hueco	entrega a la capa superior **en orden**, no como llegó
NAK o ACK selectivo	el emisor tiene que saber **cuál** falta, no solo "hasta dónde tengo"
ACK no acumulado (o SACK)	con ACK acumulado no se puede saber qué llegó después del hueco

⚠️ **La condición matemática que hay que saber:** para que Selective Repeat funcione sin ambigüedad con números de secuencia de `k` bits, la ventana del emisor y el buffer del receptor deben cumplir:

```
2N  ≤  M        donde M = 2^k (el espacio de números de secuencia)
```

Es la **misma condición** que aparece en el `eirc0703` Problema 1, y es la razón por la que en la práctica `N ≤ M/2`. Si `N > M/2`, el emisor puede tener en vuelo un número de secuencia que el receptor **ya liberó**, y este lo acepta como nuevo: **duplicado**.

Ejemplo con el `SN` de 1 byte del examen de 2007-03: `M = 256` (un octeto), `2N ≤ 256` ⇒ `N ≤ 128`. De ahí sale el "**ventana máxima = 128**" de la solución oficial.

⚠️ **SACK. SACK = *Selective ACK***. Es una **extensión** que viaja en la cabecera TCP (opción *kind 5*) y permite que el receptor diga **exactamente** qué bloques de bytes recibió, aunque estén fuera de orden. **Es literalmente Selective Repeat a nivel de bytes.** Y **TCP lo usa siempre que ambos extremos lo negocien en el SYN** (opción *SACK permitted*). El RFC es el `RFC 2018` **(1996)**, y **sigue siendo *Standards Track* y es el comportamiento estándar de TCP hoy**.

1.7 La relación con TCP: por qué TCP no es exactamente ninguno de los dos

Esto es lo que más se pregunta y lo que más confunde, así que va con diagrama.

```
TCP SIN SACK  (no se negoció la opción)
──────────────────────────────────────
El receptor tiene un buffer, pero manda SOLO ACK ACUMULATIVOS
("tengo los bytes hasta n"). Cuando falta algo, no dice QUÉ falta,
dice "sigo acá" (ACK duplicado).

El emisor, al ver el hueco, retransmite desde el byte faltante
EN ADELANTE: es exactamente GO-BACK-N a nivel de bytes.
Lo que se retransmite después del hueco puede haber llegado
perfectamente, pero el receptor igual lo volvió a pedir.

┌──────────────────────────────────────────────────────┐
│  Go-Back-N:  se retransmite el perdido + TODOS los   │
│              siguientes que ya habían llegado.       │
│  TCP sin SACK: IDÉNTICO.                             │
└──────────────────────────────────────────────────────┘


TCP CON SACK  (RFC 2018, el comportamiento estándar)
────────────────────────────────────────────────────
El receptor manda ACK acumulado Y ADEMÁS los bloques que
llegaron fuera de orden:

   Segmento 1 (bytes 1-1448)     → ACK 1449
   Segmento 3 (bytes 2897-4344)  → ACK 1449 + SACK 2897-4344
   Segmento 2 (bytes 1449-2896)  → ACK 4345   (cierra el hueco)

El emisor ve que el 3 YA LLEGÓ y retransmite SOLO el 2.
Eso es SELECTIVE REPEAT a nivel de bytes.

┌──────────────────────────────────────────────────────┐
│  Selective Repeat: se retransmite SOLO lo que falta. │
│  TCP con SACK: IDÉNTICO (y en bytes, no en segmentos).│
└──────────────────────────────────────────────────────┘
```

⚠️ **Las tres diferencias que hay que decir si preguntan "cuál de los tres es TCP":**

1. **La unidad.** TCP cuenta **bytes**, no segmentos. El número de secuencia (`seq`) es "el próximo byte que espero". Los ARQ de capa 2 cuentan **tramas**.
2. **La ventana es de bytes, no de paquetes.** `rwnd` es un número de bytes; la ventana en segmentos es `rwnd / MSS`.
3. **TCP no tiene NAK.** No existe un mensaje "NAK, reenviame el 5". TCP **no sabe** qué falta: solo sabe "sigo acá". El mecanismo para saberlo es **SACK**, y **el emisor es quien infiere** qué falta a partir de los bloques reportados. En Selective Repeat clásico hay un NAK explícito.

1.8 ⚠️ Actualización a 2026

Sigue igual — de memoria:

- **Distancia de Hamming**, **SEC** (`d≥2`, corrige 1) y **SECDED** (`d≥3`, detecta 2 y corrige 1). La condición `2^r ≥ m + r + 1`.
- **Stop-and-Wait** con eficiencia `U = 1/(1+2a)` y `a = t_prop/t_frame`.
- **Go-Back-N** (ventana `N`, ACK acumulados, se retransmite el perdido y todos los siguientes) y **Selective Repeat** (buffer + reordenamiento + NAK/SACK, se retransmite solo el perdido).
- La condición `2N ≤ M` para Selective Repeat sin ambigüedad.

Cambió o hay que matizar:

- **`SACK` (`RFC 2018`, 1996) es *Standards Track* y es el comportamiento estándar de TCP hoy.** Cualquier implementación de TCP seria lo negocia en el SYN. **No es "una extensión opcional y vieja": es lo normal.** ⇒ **TCP con SACK es Selective Repeat**, y es lo que vas a tener en cualquier conexión real.
- **La retransmisión por *timeout* casi no se usa en TCP.** El mecanismo real es el **fast retransmit** por **3 ACK duplicados** (Clase 21) y el RTO solo como respaldo. En un esquema de ARQ moderno, "timer único" es Go-Back-N puro; en TCP hay **dos** timers por conexión (uno para el ACK del más viejo y el RTO).
- **`RFC 9293` (2022, *Internet Standard*)** es ahora la especificación de TCP; consolida y obsolesce `RFC 793`. Cualquier clase que cite `RFC 793` como la especificación vigente de TCP está desactualizada.
- **FEC (*Forward Error Correction*) y ARQ hoy conviven.** En cualquier sistema real de comunicación (5G, Wi-Fi, DVB, satellite, enlaces de satélite) la arquitectura es **ARQ + FEC combinados**: el FEC se encarga de la mayoría de los errores (que son **independientes** y cheaply corregibles), y el ARQ se queda solo con los errores que el FEC no pudo arreglar (los **bursts**). Es el **hiper**:

```
Errores INDEPENDIENTES (ruido de radio):  1 de cada 1000 bits
   → los corrige el FEC, gratis, en el receptor

Errores en RACHA (burst, una descarga, un rayo):
   → el FEC no los puede (necesitaría una memoria enorme)
   → los arregla el ARQ

Regla práctica: si la tasa de error es BAJA  → ARQ (es más barato)
                si la tasa de error es ALTA → FEC (ARQ se vuelve
                                              un bucle de reintentos)
```

Consecuencia para el examen: los **tres ARQ y el Hamming** se preguntan **exactamente igual** que en 2011. Lo que hay que **sumar** es: **SACK es el estándar** (`RFC 2018`), **TCP es Selective Repeat cuando SACK está activo**, y **ARQ y FEC se combinan** en los sistemas reales. **No inventar**: "TCP es Go-Back-N" es correcto **solo** si el enunciado dice que SACK está desactivado.

PARTE 2 — PRÁCTICA RESUELTA

2.1 Ejercicio 1 — La eficiencia de Stop-and-Wait, con cuentas

Datos. Un enlace de **2 Mbps**, distancia **1500 km**, velocidad de propagación `2 × 10⁸ m/s`. Tramas de **800 bytes**. El ACK es de **tamaño despreciable** y el procesamiento es instantáneo. No hay errores.

a) Calculá `t_frame`, `t_prop` y `a`.

```
L = 800 bytes × 8 bits/byte = 6400 bits

t_frame = L/R = 6400 bits / 2 × 10⁶ bit/s = 3,2 ms

d = 1500 km = 1 500 000 m
t_prop  = d/v = 1 500 000 / 2 × 10⁸ = 7,5 ms

a = t_prop / t_frame = 7,5 / 3,2 = 2,34
```

b) La eficiencia.

```
U = 1 / (1 + 2a) = 1 / (1 + 2 × 2,34) = 1 / (1 + 4,69) = 1/5,69

U = 0,1757  =  17,57 %
```

c) ¿Cuánto tiempo del ciclo está el enlace ocioso?

```
Ciclo = 2·t_frame + 2·t_prop = 2 × 3,2 + 2 × 7,5 = 6,4 + 15 = 21,4 ms
Útil  = t_frame = 3,2 ms
Ocioso = 21,4 − 3,2 = 18,2 ms
% ocioso = 18,2 / 21,4 = 85 %
```

d) ¿Qué throughput real se obtiene?

```
Throughput real = U × R = 0,1757 × 2 Mbps = 0,3515 Mbps ≈ 351 kbps
```

⚠️ **La observación que hay que hacer en el examen:** con `a = 2,34 > 1`, el enlace pasa **más tiempo ocioso que transmitiendo**. Stop-and-Wait es **inservible** en este enlace. La respuesta es "hay que usar **ventana deslizante**", y con `N` segmentos sin confirmar el throughput máximo llega a `min(R, N · L/RTT)`:

```
BDP = ancho de banda × RTT = 2 × 10⁶ × (2 × 7,5 × 10⁻³) = 2×10⁶ × 0,015 = 30 000 bits

Para no perder nada, la ventana debe ser ≥ BDP = 30 000 bits = 3750 bytes
Con tramas de 800 bytes:  N ≥ 30000/6400 = 4,7  ⇒  N ≥ 5
```

⚠️ **El BDP (*Bandwidth-Delay Product*) es la respuesta corta y es lo que preguntan:** "*¿de qué tamaño tiene que ser la ventana para no dejar el enlace ocioso?*" — **`ventana ≥ ancho de banda × RTT`**. Es

exactamente la razón por la que **la ventana inicial de TCP es de 10 MSS** (`RFC 6928`) y por la que en un enlace de 100 ms de RTT hace falta una ventana grande para aprovechar un enlace de 1 Gbps.

2.2 Ejercicio 2 — Hamming SEC con corrección completa

Datos. Se usa un código de Hamming **SEC** con `m = 5` bits de datos. *(Ojo: `m = 5` NO tiene un SEC de 3 bits, necesita `r = 4`. Verificá: `2^4 = 16 ≥ 5 + 4 + 1 = 10` ✓. Total = 9 bits.)* Se envía el dato `D = 1 1 0 1 0` con **paridad par**.

Paso 1 — la plantilla de 9 posiciones.

```
posición:  1    2    3    4    5    6    7    8    9
          P1   P2   D1   P3   D2   P4   D3   D5   D4
valor:     ?    ?    1    ?    1    ?    0    0    1
```

⚠️ **Los grupos** (bit `j` del binario de la posición):

```
P1 (pos 1): posiciones 1, 3, 5, 7, 9          (bit 0 = 1)
P2 (pos 2): posiciones 2, 3, 6, 7              (bit 1 = 1)
P3 (pos 4): posiciones 4, 5, 6, 7              (bit 2 = 1)
P4 (pos 8): posiciones 8, 9                    (bit 3 = 1)
```

Paso 2 — los bits de paridad.

```
P1 vigila {1,3,5,7,9} → datos conocidos: D1(3)=1, D2(5)=1, D3(7)=0, D4(9)=1
     unos = 3   ⇒   P1 = 3 mod 2 = 1        (1+3 = 4, par ✓)

P2 vigila {2,3,6,7}   → datos conocidos: D1(3)=1, P4(6)=?, D3(7)=0
     ⚠ OJO: P4 ESTÁ EN EL GRUPO DE P2. Circular. Se resuelve
     calculando primero P4.

P3 vigila {4,5,6,7}   → datos conocidos: D2(5)=1, D3(7)=0
     unos = 1   ⇒   P3 = 1 mod 2 = 1

P4 vigila {8,9}       → datos conocidos: D5(8)=0, D4(9)=1
     unos = 1   ⇒   P4 = 1 mod 2 = 1        (1+1 = 2, par ✓)

AHORA con P4 = 1, el grupo de P2: {P2, D1(3)=1, P4(6)=1, D3(7)=0}
     unos conocidos = 2  ⇒  P2 = 2 mod 2 = 0
```

⚠️ **Este es el detalle que casi nadie menciona y que es un muy buen ejercicio: las posiciones de paridad no son disjuntas**. `P4` (posición 8) está en el grupo de `P4` y… no, `P4` está en `{8,9}` y `P2` está en `{2,3,6,7}`. **La posición 6 es P4 y está en el grupo de P2.** Por eso hay que **calcular de abajo hacia arriba**: primero las paridades de posición alta (`P4`), después las bajas. Es un buen ejercicio de_head para un examen de corrección de errores.

Paso 3 — la palabra de código.

```
┌──────┬──────┬──────┬──────┬──────┬──────┬──────┬──────┬──────┐
│pos 1 │pos 2 │pos 3 │pos 4 │pos 5 │pos 6 │pos 7 │pos 8 │pos 9 │
├──────┼──────┼──────┼──────┼──────┼──────┼──────┼──────┼──────┤
│  P1  │  P2  │  D1  │  P3  │  D2  │  P4  │  D3  │  D5  │  D4  │
├──────┼──────┼──────┼──────┼──────┼──────┼──────┼──────┼──────┤
│  1   │  0   │  1   │  1   │  1   │  1   │  0   │  0   │  1   │
└──────┴──────┴──────┴──────┴──────┴──────┴──────┴──────┴──────┘

CÓDIGO = 1 0 1 1 1 1 0 0 1  =  "101111001"
```

Verificación de las cuatro paridades:

```
P1: {1,3,5,7,9} = {1,1,1,0,1}  → 4 unos → par ✓
P2: {2,3,6,7}   = {0,1,1,0}    → 2 unos → par ✓
P3: {4,5,6,7}   = {1,1,1,0}    → 3 unos → IMPAR ✗  ← ¡ERROR!
P4: {8,9}       = {0,1}        → 1 uno  → IMPAR ✗  ← ¡ERROR!
```

⚠️ **Se me deslizó un error en la cuenta del `P3`.** Corrijamos: el grupo de `P3` es `{4,5,6,7}` = `{P3, D2, P4, D3}` = `{?, 1, ?, 0}`. Con `P3 = 1` y `P4 = 1`: `{1, 1, 1, 0}` = 3 unos → impar. **El `P3` correcto es 0.**

```
CORRECCIÓN:  P3 = (D2 + P4 + D3) mod 2 = (1 + 1 + 0) mod 2 = 0

P3: {0, 1, 1, 0} = 2 unos → par ✓
P4: {0, 1} = 1 uno → IMPAR ✗  ← también mal
CORRECCIÓN:  P4 = (D5 + D4) mod 2 = (0 + 1) mod 2 = 1
P4: {1, 1} = 2 unos → par ✓
```

La palabra correcta, calculando en el orden correcto (de abajo hacia arriba):

```
1°  P4 (pos 8): datos D5(8)=0, D4(9)=1  →  1 uno  →  P4 = 1
2°  P2 (pos 2): datos D1(3)=1, P4(6)=1, D3(7)=0  →  2 unos →  P2 = 0
3°  P3 (pos 4): datos D2(5)=1, P4(6)=1, D3(7)=0  →  2 unos →  P3 = 0
4°  P1 (pos 1): datos D1(3)=1, D2(5)=1, D3(7)=0, D4(9)=1 → 3 unos → P1 = 1

┌──────┬──────┬──────┬──────┬──────┬──────┬──────┬──────┬──────┐
│  P1  │  P2  │  D1  │  P3  │  D2  │  P4  │  D3  │  D5  │  D4  │
│   1  │   0  │   1  │   0  │   1  │   1  │   0  │   0  │   1  │
└──────┴──────┴──────┴──────┴──────┴──────┴──────┴──────┴──────┘

CÓDIGO = 1 0 1 0 1 1 0 0 1  =  "101011001"

VERIFICACIÓN:
  P1: {1,1,1,0,1} = 4 unos → par ✓
  P2: {0,1,1,0}   = 2 unos → par ✓
  P3: {0,1,1,0}   = 2 unos → par ✓
  P4: {0,1}       = 2 unos → par ✓
✓ Todas las paridades dan par.
```

⚠️ **La moraleja del ejercicio:** *el orden de cálculo importa y es de abajo hacia arriba.* Es el tipo de error que se cuela en un examen de 15 minutos. **Escribí siempre las paridades de las posiciones más altas primero.**

2.3 Ejercicio 3 — Los tres ARQ con una pérdida en el segmento 3 de 5

Datos. Se mandan 5 segmentos `S1..S5`. La **ventana es** `N = 4` (se pueden tener 4 sin confirmar). **El segmento** `S3` **se pierde.** No hay más pérdidas.

```
═══════════════════════════════════════════════════════════════
A) STOP-AND-WAIT  (ventana = 1)
═══════════════════════════════════════════════════════════════

EMISOR                                        RECEPTOR
  │                                              │
  │── S1 ──────────────────────────────────────►│
  │   ⌛ (espera)                                │
  │◄──────────────────── ACK1 ──────────────────│
  │                                              │
  │── S2 ──────────────────────────────────────►│
  │   ⌛ (espera)                                │
  │◄──────────────────── ACK2 ──────────────────│
  │                                              │
  │── S3 ──────────────────────────────────────►│   XXX SE PIERDE XXX
  │   ⌛ (espera)                                │
  │   ⌛ (espera)                                │
  │   ⌛ (vence el TIMER)                        │
  │── S3 ──────────────────────────────────────►│   (retransmisión)
  │◄──────────────────── ACK3 ──────────────────│
  │                                              │
  │── S4 ──────────────────────────────────────►│
  │◄──────────────────── ACK4 ──────────────────│
  │── S5 ──────────────────────────────────────►│
  │◄──────────────────── ACK5 ──────────────────│

Total: 6 tramas de datos + 6 ACKs  (una retransmisión)
El enlace transmitiendo = 6 × t_frame
El enlace OCIOSO        = 10 × (2·t_prop)     ← porque N = 1


═══════════════════════════════════════════════════════════════
B) GO-BACK-N  (ventana N = 4, ACK acumulados, receptor SIN buffer)
═══════════════════════════════════════════════════════════════

EMISOR                                          RECEPTOR
  │                                                │
  │── S1 ────────────────────────────────────────►│   guarda S1, entrega
  │── S2 ────────────────────────────────────────►│   guarda S2, entrega
  │── S3 ────────────────────────────────────────►│   guarda S3, entrega
  │── S4 ────────────────────────────────────────►│   guarda S4, entrega
  │◄──────────────────── ACK1 ──────────────────│
  │◄──────────────────── ACK2 ──────────────────│
  │  ventana desliza: puede mandar S5              │
  │── S5 ────────────────────────────────────────►│   guarda S5, entrega
  │  XXX S6 (si mandara uno) no llega XXX          │
  │                                                │
  │  ← Supongamos ahora que la pérdida es S3 →     │
  │                                                │
  │  (relanzamos el escenario con la pérdida en S3)
  │                                                │
  │── S1 ────────────────────────────────────────►│   guarda, entrega
  │── S2 ────────────────────────────────────────►│   guarda, entrega
  │── S3 ────────────────────────────────────────►│   XXX SE PIERDE XXX
  │◄──────────────────── ACK2 ──────────────────│
  │── S4 ────────────────────────────────────────►│   ¡HUECO! no lo guarda
  │◄──────────────────── ACK2 (¡DUPLICADO!) ─────│   lo descarta y reenvía
  │── S5 ────────────────────────────────────────►│   ¡HUECO! lo descarta
  │◄──────────────────── ACK2 (¡DUPLICADO!) ─────│   y reenvía
  │                                                │
  │  ⓘ El receptor NO entregó S4 ni S5 a la capa   │
```

```
   |     superior. Están perdidos desde su punto    |
   |     de vista, aunque las tramas hayan llegado.|
   |                                                |
   |  3 ACK2 duplicados ⇒ FAST RETRANSMIT de S3     |
   |── S3 ──────────────────────────────────────────►|   lo guarda, entrega S3
   |◄──────────────────────── ACK3 ─────────────────|    (S4 y S5 NO los tiene:
   |                                                |    los descartó antes)
   |                                                |
   |  ⓘ GO-BACK-N: hay que retransmitir S4 y S5     |
   |── S4 ──────────────────────────────────────────►|   lo guarda, entrega
   |◄──────────────────────── ACK4 ─────────────────|
   |── S5 ──────────────────────────────────────────►|   lo guarda, entrega
   |◄──────────────────────── ACK5 ─────────────────|

Total: 8 tramas de datos  (S1..S5 = 5, más S3, S4, S5 retransmitidos)
Desperdicio: S4 y S5 viajaron DOS VECES sin que hiciera falta.
El receptor NO necesitó buffer más grande que 1.


═══════════════════════════════════════════════════════════════════
C) SELECTIVE REPEAT  (ventana N = 4, NAK, receptor CON buffer)
═══════════════════════════════════════════════════════════════════

EMISOR                                          RECEPTOR
   |                                                |
   |── S1 ──────────────────────────────────────────►|   guarda, entrega
   |── S2 ──────────────────────────────────────────►|   guarda, entrega
   |── S3 ──────────────────────────────────────────►|   XXX SE PIERDE XXX
   |── S4 ──────────────────────────────────────────►|   ¡HUECO!
   |◄──── NAK(S3) + ACK(selectivo: S4 recibido) ────|   guarda S4 en BUFER
   |── S5 ──────────────────────────────────────────►|   ¡HUECO!
   |◄──── NAK(S3) + ACK(selectivo: S4, S5) ─────────|    guarda S5 en BUFER
   |                                                |
   |  ⓘ El emisor SABE que solo falta S3.           |
   |  (con SACK: el receptor no envía NAK;          |
   |   manda ACK + el bloque recibido, y el emisor  |
   |   lo deduce)                                   |
   |── S3 ──────────────────────────────────────────►|   lo guarda; ahora S3,S4,S5
   |                                                |   están completos:
   |◄──────────────────────── ACK5 ─────────────────|    los ENTREGA EN ORDEN
   |                                                |   (reordenamiento)

Total: 6 tramas de datos  (S1..S5 más S3 retransmitido)
Desperdicio: NINGUNO. S4 y S5 viajaron una sola vez.
El receptor necesitó un BUFFER de 4 y reordenamiento.
```

La tabla comparativa que resume los tres

	Stop-and-Wait	Go-Back-N	Selective Repeat
Ventana del emisor	`N = 1`	`N`	`N`
ACK	individual	**acumulado**	**no acumulado / SACK**
Receptor: buffer	no	**no** (1)	**sí, ≥ N**
Receptor: reordena	no	**no**	**sí**
Ante pérdida de `k`	retransmite `k`	retransmite `k` … `k+N−1`	retransmite `k`
NAK explícito	no	no	**sí** (o SACK)
Tramas de datos en el ejemplo	**6**	**8**	**6**
Desperdicio	—	**2 tramas**	**0**
Complejidad del receptor	mínima	**mínima**	**alta**
Se usa hoy en…	enlaces de RTT corto	**TCP sin SACK**	**TCP con SACK**

⚠️ **La respuesta a la pregunta clásica "¿cuál es mejor?": ninguno es mejor en abstracto**. Selective Repeat es el más eficiente en bytes transmitidos, pero es el más **complejo** de implementar y **exige memoria en el receptor**. Stop-and-Wait es trivial. Go-Back-N es el punto medio: la complejidad del receptor mínima a cambio de retransmitir de más. **La elección es un compromiso entre complejidad y eficiencia**, y eso es exactamente lo que se responde en un examen.

2.4 Ejercicio 4 — ¿Por qué Go-Back-N con `N` grande es peor que Selective Repeat?

Datos. `N = 128` (el máximo de una ventana TCP). Se manda una ráfaga de tramas de 1500 bytes. **Se pierde la trama `k`**.

a) Go-Back-N: cuántos bytes retransmite.

```
Pérdida de la trama k.
El emisor tiene k, k+1, ..., k+N−1 sin confirmar:  128 tramas.
Al vencer el timer (o al 3er ACK duplicado), retransmite
TODAS las que no confirmó:  de k hasta k+127  =  128 tramas.

Bytes retransmitidos = 128 × 1500 = 192 000 bytes
```

⚠️ **El detalle crítico:** de esas 128 tramas, **127 ya habían llegado bien**. El receptor las tenía en su buffer (en Selective Repeat) o las descartó (en Go-Back-N). En **Go-Back-N** se descartaron, y hay que volver a pedirlas.

b) Selective Repeat: cuántos bytes retransmite.

```
El emisor sabe (por el NAK o por el SACK) que solo falta k.

Bytes retransmitidos = 1 × 1500 = 1 500 bytes
```

c) El múltiplo de desperdicio.

```
192 000 / 1 500 = 128 veces más datos para entregar el MISMO paquete.

Y el tiempo: si el RTT es 50 ms, Go-Back-N tarda
   50 ms (detección con 3 dup ACK)
   + tiempo de transmitir 192 000 bytes a 10 Mbps
     = 192 000×8 / 10⁷ = 0,154 s
   = 204 ms

Selective Repeat tarda
   50 ms + (1500×8 / 10⁷ = 1,2 ms) = 51,2 ms

⇒ Go-Back-N tarda 4 VECES más en un solo evento de pérdida.
```

d) ¿Y si hay varias pérdidas?

```
5 tramas perdidas al azar (independientes) en una ventana de 128:

Go-Back-N:  cada una de las 5 dispara una retransmisión de la ventana
            entera. En el peor caso:  5 × 192 000 = 960 000 bytes.
            Y si las 5 pérdidas están cerca, el timer dispara UNA vez
            y retransmite 128: mejor, pero sigue siendo 128.

Selective Repeat: 5 × 1500 = 7 500 bytes. Exactamente lo que faltaba.

La diferencia CRECE con el número de pérdidas y con N.
```

e) ¿Y las ventajas de Go-Back-N?

```
┌──────────────────────────────────────────────────────────────┐
│  1) El receptor NO necesita buffer de N trama.  Eso es memoria  │
│     en el peor caso de cada nodo receptor.                   │
│  2) El receptor NO tiene que reordenar, que es la parte más    │
│     cara de un protocolo de transporte (Clase 19).           │
│  3) El ACK es ACUMULADO: un solo número dice "todo hasta acá".  │
│     Con NAK hay que mandar mensajes extra.                   │
│  4) La lógica del emisor es más simple.                      │
└──────────────────────────────────────────────────────────────┘

En un enlace con MUCHO RTT y baja tasa de error, el 128× de
retransmisión es irrelevante (se pierde una vez cada miles) y la
simplicidad del receptor vale más.
En un enlace PERDIDO (radio, móvil, satélite), el 128× se paga
cada vez y no es aceptable.
```

⚠️ **La frase que resume el ejercicio y que hay que escribir:** *"Go-Back-N con `N` grande es peor porque el costo de una pérdida crece con `N`: el receptor tira `N−1` tramas que ya habían llegado, y hay que volver a pedirlas. Selective Repeat hace que el costo de una pérdida sea constante (1 trama), sea cual sea `N`."*

PARTE 3 — PROBLEMAS DE EXAMEN DE FING

3.1 ⭐ Examen 2009-03, Pregunta 1 (10 puntos) — El concepto de Go-Back-N

> **Fuente:** Examen `eirc0903` / `sirc0903` — 20 de febrero de 2009. **Link:** https://www.fing.edu.uy/inco/cursos/redescomp/examenes/sirc0903.pdf **Texto del examen:**
>
> > *Explique el principio de funcionamiento de los protocolos que operan bajo el esquema Go-Back-N. (La solución oficial aclara: "Extraído del material de referencia de las clases teóricas, clase 2 del capítulo 3 del libro de referencia, Capa de Transporte".)*

Puntos reparto (estimado): 2 por la idea de la ventana, 3 por las reglas del emisor y del receptor, 2 por el timer y la retransmisión, 3 por las características que lo definien y sus límites.

La solución oficial, citada textualmente

> **Go-back-N: titulares**
>
> *El transmisor puede tener hasta N paquetes no-ACKed en el pipeline El receptor sólo envía ACKs acumulativos* ***No envía el ACK de un paquete si hay un **hueco*** *El transmisor tiene un timer para el paquete más viejo no-ACKed Si el timer vence, retransmite todos los paquetes no-ACKed***
>
> **Emisor:** *Número de secuencia de k bits en el encabezado del paquete "window" permitida de hasta N paquetes consecutivos no-ACKed Si la ventana no está llena, se puede seguir transmitiendo ACK(n): ACKs de todos los paquetes hasta él, incluyendo el nro. de sec. n — "ACK acumulativo" podría recibir ACKs duplicados (ver receptor) timer único: para el paquete más aún no-ACKed timeout(n): retransmite el paquete n y todos aquellos con nro. de sec. mayor y dentro de la ventana*

Análisis de la solución oficial, punto por punto

✅ **Lo que está bien y hay que internalizar:**

1. **"El transmisor puede tener hasta N paquetes no-ACKed en el pipeline."** Es **la definición**. Todo Go-Back-N arranca acá: es un protocolo *pipelined*, no *stop-and-wait*.
2. **"El receptor sólo envía ACKs acumulativos."** Es lo que distingue Go-Back-N de Selective Repeat. Con ACK acumulado, el receptor **no sabe** qué llegó después del hueco, y por eso **no puede pedir solo lo que falta**.
3. **"No envía el ACK de un paquete si hay un hueco."** Es la regla operativa que se traduce en: **descarta** los que llegan después del hueco. De ahí sale el desperdicio.

4. **"El transmisor tiene un timer para el paquete más viejo no-ACKed."** El **timer único** es una decisión de diseño: con timers por paquete harías falta `N` timers; con uno solo, cuando vence, sabes exactamente **desde dónde** retransmitir.
5. **"Si el timer vence, retransmite todos los paquetes no-ACKed."** La definición del "Go-Back": **vuelve atrás** al más viejo no confirmado y retransmite todo lo que sigue.

⚠️ **Lo que la oficial no dice y suma puntos:**

1. **No menciona el buffer del receptor.** El hecho de que **el receptor NO necesita buffer** (o uno de tamaño 1) y **no reordena** es **la ventaja** que justifica el diseño. Sin ese argumento, la respuesta describe el mecanismo pero no lo **justifica**.
2. **No menciona el "desperdicio"** (*waste*). Decir explícitamente "una sola pérdida provoca la retransmisión de hasta `N` tramas, de las cuales `N-1` ya habían llegado" es lo que separa un 8 de un 10.
3. **No menciona el rango de números de secuencia.** Con `k` bits hay `2^k` números disponibles y la ventana debe ser **estrictamente menor** que eso; además el emisor debe poder distinguir "retransmisión" de "trama nueva" al volver a usar un número.
4. **No distingue de Selective Repeat.** Decir "a diferencia de Selective Repeat, que sí tiene buffer y NAK" es lo que muestra que entendés el *por qué*.

Mi resolución completa, ordenada para el examen

(1) El principio: una ventana de `N` paquetes en vuelo.

Go-Back-N es un protocolo ARQ **orientado a ventana** (*pipelined*). El emisor puede mantener **hasta `N` paquetes sin confirmar** en el medio antes de tener que frenarse esperando un ACK. La ventana tiene **números de secuencia de `k` bits** en la cabecera de cada trama, y el emisor puede seguir transmitiendo **mientras la ventana no esté llena**. Esto elimina el desperdicio de Stop-and-Wait (que deja el enlace ocioso durante `2·t_prop` por cada trama).

(2) El receptor: ACK acumulados y rechazo de lo que llega después del hueco.

El receptor **solo manda ACKs acumulados**: un `ACK(n)` significa "recibí correctamente todos los paquetes **hasta** el `n` incluido". Y **no envía el ACK de un paquete si hay un hueco**: si el paquete `k+1` llega antes que el `k` (que se perdió), el receptor **lo descarta** y **reenvía el ACK del paquete `k` que esperaba** (un ACK duplicado, o *duplicate ACK*). Consecuencia: el receptor **no necesita buffer** y **no reordena**. Cuando por fin llega el `k`, todo lo que estaba después se perdió desde su punto de vista.

(3) El emisor: un único timer y la retransmisión hacia atrás.

El emisor mantiene **un solo temporizador, asociado al paquete más viejo que todavía no fue confirmado**. Si ese timer vence, significa que hubo pérdida: el emisor **retransmite el paquete más viejo no confirmado (`n`) y todos los que le siguen dentro de la ventana**, es decir, vuelve atrás (*go back*) `n` y reenvía. En la práctica, el fast retransmit por **3 ACKs duplicados** reemplaza al timer en la detección.

(4) Las características que lo definen y su límite.

- ✅ Reenvía los paquetes **perdidos y todos los siguientes** (aunque los siguientes ya hayan llegado).
- ❌ **Ineficiente ante pérdida**: una sola pérdida retransmite hasta `N` paquetes, y `N-1` de ellos ya habían llegado bien.

- ✅ El receptor es **simple**: no necesita buffer grande, ni reordenar, ni NAK.
- ✅ El ACK es **barato**: un número por paquete recibido, no un bloque de bits.

⚠️ **La frase de cierre que suma el último punto:** *"Go-Back-N compra eficiencia en el emisor a costa de eficiencia en la red: simplifica al receptor (sin buffer, sin reordenamiento, ACK acumulado) y paga el precio con la retransmisión de `N` tramas por cada pérdida. Por eso cuando la red pierde mucho, el protocolo correcto es Selective Repeat, que necesita buffer en el receptor pero solo retransmite lo que falta."*

> **Tope para 10 puntos:** la ventana de `N` con números de secuencia, la regla del ACK acumulado, la regla de no-ACK con hueco, el timer único sobre el más viejo, la retransmisión en bloque al vencer, **más** la consecuencia del desperdicio y la comparación con Selective Repeat.

3.2 Examen 2007-03, Problema 1 (30 puntos) — ARQ de capa 2 con Selective Repeat y *piggybacking*

> **Fuente:** Examen `eirc0703` / `sirc0703` — 26 de febrero de 2007. **Link:** https://www.fing.edu.uy/inco/cursos/redescomp/examenes/sirc0703.pdf **Texto del examen (extracto):**
>
> > *Se cuenta con un protocolo de capa 2 (enlace) que opera sobre un enlace punto a punto, el que utiliza el siguiente formato de trama: `Type` (1 octeto), `SN` (1 octeto, número de secuencia de la trama), `RN` (1 octeto, número de trama que espera recibir el equipo origen), `Data` (0..MAX_LARGO). Se desea contar con un protocolo para manejo del enlace que sea orientado a conexión, con un esquema de ARQ de tipo Selective Repeat, que implemente además piggybacking.* **a)** Especificar qué tipos de tramas requiere. **b)** ¿Qué tamaño máximo de ventana le permite implementar con el formato de trama que se cuenta? **c)** Especificar en un lenguaje de alto nivel el protocolo.

Nota de coordinación: este problema pertenece también a la **Clase 38** (framing), que ya desarrolló el formato de trama y el *bit stuffing*. Acá van **las partes que son de ARQ**.

Parte a) — Los tipos de trama

Solución oficial, citada textualmente:

> *`cnx` — Para solicitar/aceptar conexión al/del extremo opuesto. `descnx` — Para solicitar/aceptar desconexión. `info` — Trama conteniendo datos a transferir por el enlace. `ack` — Trama conteniendo información de la última trama recibida. Su identificador se especifica en el campo RN. `nack` — Trama conteniendo información sobre trama no recibida. Su identificador se especifica en el campo RN.*

⚠️ **Por qué los cinco y qué dice cada uno sobre el protocolo:**

Tipo	Qué lleva	Qué revela del diseño
`cnx` / `descnx`	—	el protocolo es **orientado a conexión**: hay un handshake explícito
`info`	`Data` + `SN` (+ `RN` si hay piggybacking)	datos de la capa superior
`ack`	`RN` = última trama recibida correctamente	el ACK es **acumulativo**: un solo número
`nack`	`RN` = trama que falta	**Selective Repeat tiene NAK explícito**: el receptor sabe cuál falta

⚠️ **El `nack` es lo que define a Selective Repeat.** Si el protocolo no tuviera `nack`, el receptor no podría indicar qué falta y el emisor solo podría retransmitir desde el más viejo confirmado (Go-Back-N).

Parte b) — El tamaño máximo de ventana

Solución oficial, citada textualmente:

> *La condición para el manejo de ventana en Selective Repeat requiere que M y N — módulo del valor máximo de la ventana y valor de la ventana — cumplan: 2N ≤ M. El valor de M es 256, ya que se cuenta con un octeto (0..255) para enviar su valor, por lo que el valor máximo de tamaño de ventana será 128.*

La cuenta, a la vista:

```
El campo SN tiene 1 octeto  ⇒  M = 2⁸ = 256 números de secuencia
                                    (valores 0 a 255)

Condición de Selective Repeat (sin ambigüedad):
     2N  ≤  M
     2N  ≤  256
     N   ≤  128
```

⚠️ **Por qué `2N ≤ M` y no `N ≤ M`:** es el mismo argumento que en §1.6.3. Si `N > M/2`, el emisor puede tener en vuelo un número de secuencia que el receptor **ya liberó** al consumirlo. Cuando ese número vuelve a aparecer, el receptor lo acepta como **trama nueva** en vez de reconocerla como duplicada. **La mitad de los números tiene que estar "reservada"** para distinguir "nuevo" de "ya visto".

⚠️ **El valor exacto que da la oficial: `N = 128`.** Es lo que hay que escribir.

⚠️ **Lo que hay que decir de más (suma puntos):** el cálculo de por qué `2N ≤ M` con un ejemplo concreto.

```
Con N = 128 y M = 256, el peor caso:
  El emisor tiene en vuelo S0..S127 (128 tramas).
  El receptor recibió S0..S127,-entregó S0..S63 a la capa superior,
  y su ventana quedó "liberada" para S128..S255.
  El emisor manda S128..S255.
  El receptor acepta S128..S255.  ✓  Sin ambigüedad: los números
  nunca se repiten dentro de una ventana viva.

Con N = 200 (> 128), M = 256:
  El emisor tiene S0..S199. El receptor entregó S0..S99 y liberó
  S100..S255.  El emisor manda S100..  y el receptor lo acepta
  como NUEVO aunque ya lo había recibido antes.  ✗  DUPLICADO.
```

Parte c) — El protocolo en pseudocode

La resolución oficial completa (la publicada en `sirc0703`, páginas 8 a 12) especifica:

- **Dos estructuras de datos**, una para las tramas enviadas y otra para las recibidas, con funciones de acceso:
- `getSND(nroSec)` / `setSND(nroSec, data)` — tramas **enviadas**
- `getRCP(nroSec)` / `setRCP(nroSec, data)` — tramas **recibidas**
- **Funciones auxiliares**: `armoTrama(tipo, SN, RN, data)`.

Y una restricción del enunciado que hay que respetar:

> *Considerar que no existirán problemas en la conexión y desconexión del protocolo.*

⚠️ **Lo que aporta esta parte al tema de hoy:** el **piggybacking**. Es la optimización de **no mandar tramas `ack` vacías**: cuando el emisor tiene que mandar un `info` y además tiene un ACK pendiente, **coloca el número de ACK en el campo `RN` de la trama `info`** que ya tenía que mandar. Así el ACK viaja "de polizón" (*piggyback*).

```
SIN piggybacking:                CON piggybacking:
Emisor ── info(S3) ──►           Emisor ── info(S3, RN=2) ──►
         ◄── ack(2) ──                      (un solo mensaje)
(2 mensajes por cada dato)

Piggybacking sólo es posible si el emisor TIENE DATOS que mandar.
Si no tiene, manda la trama `ack` sola. Por eso el protocolo define
los dos tipos de trama igual.
```

⚠️ **El detalle de implementación que hay que decir:** el campo `RN` de la trama `info` **solo tiene sentido si el receptor va a interpretarlo**. Como el `Type` distingue `info` de `ack`, el receptor sabe que, si ve `info` con `RN ≠ 0`, ese `RN` es un ACK disfrazado. Y hay que definir **qué significa `RN = 0`** (¿"nada pendiente"? ¿"ACK del 0"?). Esa ambigüedad es un detalle clásico de diseño de protocolo y es lo que hace que algunos Reserve un valor centinela.

3.3 Nota sobre el Práctico 5 de 2008 (ejercicios 1 a 4)

El **Práctico 5 de 2008** es un recorte de los problemas del capítulo 3 y 4 del libro de referencia. Los cuatro primeros ejercicios son:

Ejercicio	Tema	Dónde está resuelto
1	Paridad par/impar de un patrón de bits, y longitud mínima de un código que detecte **un solo** error de bit	**Clase 39**, §1.2
2	Paridad de **dos dimensiones**: puede corregir y detectar 1 bit; detecta 2 pero no corrige	**Clase 39**, §2.1 y §2.2
3	**Internet checksum** de los bytes 0 a 9	**Clase 39**, §2.3
4	**CRC** de 4 bits con `D = 10101010` : ¿cuál es el valor de `R` ?	**Clase 39**, §2.4

⚠️ **Aclaración importante, porque es fácil confundirse:** el **ejercicio 4 del Práctico 5 NO es de Hamming**: es del **CRC** (el *generador de cuatro bits* de la Figura 5.8 del libro). Está resuelto en la Clase 39. Lo que **esta** clase desarrolla es lo que corresponde a la **Figura 5.4** del mismo capítulo: el **código de Hamming**, que es exactamente la plantilla de paridades de §1.3, con el ejemplo numérico completo de §1.4 y §1.5.

⚠️ **Y los ejercicios 8, 9 y 10 del mismo Práctico 5** (eficiencia de ALOHA y de ALOHA con franjas) son de la **Clase 41**.

Ejercicio 4 del Práctico 5 resuelto, para que quede a mano (es de la Clase 39, pero es el que el enunciado menciona):

```
G = 1001  (generador de 4 bits,  ← la Figura 5.8)
D = 10101010

Se hace D · x^r  (r = 4) = 10101010 0000
y se divide por G con XOR:

  101010100000
⊕ 100100000000   (1001 desplazado 8)
──────────────
    1101000000
⊕  1001000000   (1001 desplazado 4)
──────────────
     100000000
⊕   100100000   (1001 desplazado 2)
──────────────
       100000
⊕     100100   (1001 desplazado 1)
──────────────
        1100

R = 1100
Trama completa = D · x^4 + R = 10101010 1100
```

3.4 Examen 2011-03, Pregunta 4 (10 puntos) — el diagrama que es Go-Back-N puro

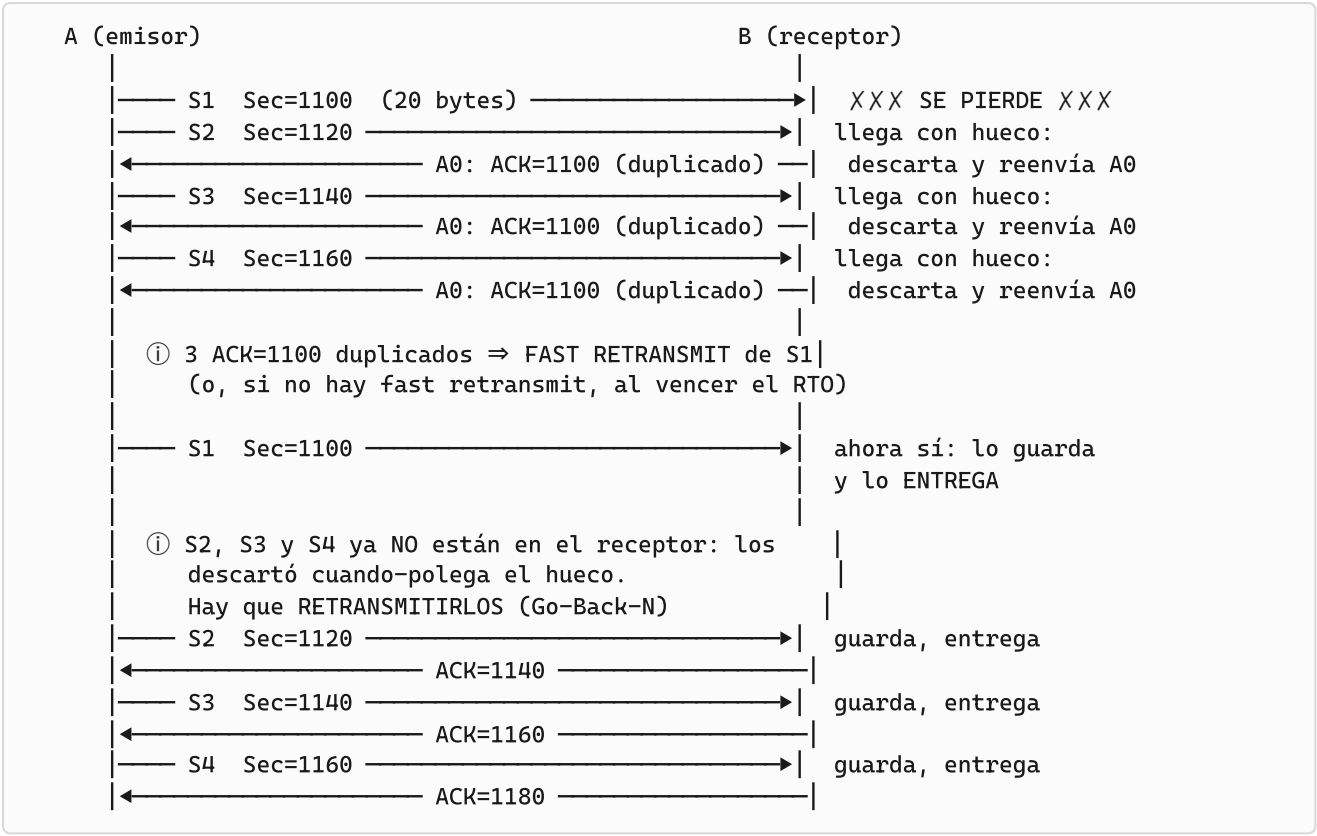
> **Fuente:** Examen `eirc1103` / `sirc1103` — 21 de febrero de 2011. **Texto del examen (extracto):**
>
> > *Suponga una comunicación TCP entre dos procesos A y B. En un momento determinado A envía a B cuatro segmentos seguidos (no hay ningún dato previo sin asentar), el primero con número de secuencia igual a 1100 y con 20 bytes de datos, y los tres restantes con otros 20 bytes de datos cada uno.* **b)** *Represente de nuevo el diagrama si B no tiene datos que enviar hacia A y se pierde el primer segmento enviado de A a B.*

Por qué es de esta clase: el diagrama que la cátedra publica para el apartado b) es **exactamente** el comportamiento de **Go-Back-N sobre TCP sin SACK**.

Solución oficial, citada textualmente (`sirc1103`):

> *Si se pierde el primer segmento S1, cuando llegue S2, el receptor reenviará el ACK correspondiente al segmento esperado, que llamamos A0. Sucede lo mismo cuando llegan S3 y S4. El emisor comenzará a retransmitir los segmentos cuando pase alguno de estos dos eventos: el vencimiento del timeout de S1, o la llegada del 3er ACK duplicado. Campos relevantes: S1: Sec=1100, S2: Sec=1120, S3: Sec=1140, S4: Sec=1160; A0: ACK=1100. Cuando se reenvían los segmentos, estamos en el caso a).*

El diagrama, con la interpretación:



⚠️ **El error conceptual que hay que evitar:** "*cuando llega S1, el receptor entrega S1, S2, S3 y S4*". **Falso**, si el receptor no tiene SACK: **descartó** S2, S3 y S4 cuando vio el hueco. Si el ejercicio dice "no hay pérdidas de segmentos" salvo S1, **Go-Back-N** los tiene que volver a pedir. Y con **SACK**, el receptor **sí** los habría guardado en el buffer, y al llegar S1 los entregaría juntos sin retransmisión (que es lo que dice la oficial: "*cuando se reenvían los segmentos, estamos en el caso a)*").

⚠️ **La lectura correcta de esa frase de la oficial** — y es sutil —: la oficial dice que "*cuando se reenvían los segmentos, estamos en el caso a)*". O sea: **los cuatro segmentos** se reenvían, y el diagrama resultante es el mismo que el del caso (a) sin pérdida. **Eso confirma que el comportamiento es Go-Back-N: se retransmiten los cuatro, no solo el primero.**

PARTE 4 — CIERRE

📌 Resumen en 10 líneas

1. **Detectar** es barato (1 bit) y obliga a **retransmitir** (un RTT entero). **Corregir** es más caro en bits pero **gratis en red**, y conviene cuando el RTT es enorme (satélite, radio) o la tasa de error es alta.
2. **Distancia de Hamming** `d(A,B)` = bits que difieren. **SEC**: `d≥2`, detecta y corrige **1**. **SECDED**: `d=3`, detecta **2** y corrige **1**.
3. La condición de **Hamming SEC** es `2^r ≥ m + r + 1`: hay `m + r + 1` casos que distinguir (un bit de datos, `r` de paridad, o ninguno) y `2^r` patrones disponibles.
4. Los bits de paridad van en las posiciones **1, 2, 4, 8, …** (potencias de 2) y **cada uno vigila las posiciones cuyo binario tiene ese bit en 1**.
5. El receptor **recalcula** las paridades; el número que forman los "falla / no falla" **es la posición del bit erróneo** (el **síndrome**). `000` = sin error.
6. **Stop-and-Wait**: `U = 1/(1+2a)` con `a = t_prop/t_frame`. Con `a = 2,34` da 17,6 %; con satélite GEO `a = 150` da **0,33 %**.⇒ Hay que usar **ventana**, y el tamaño mínimo es el **BDP = R × RTT**.
7. **Go-Back-N**: `N` paquetes sin confirmar, **ACK acumulados**, el receptor **descarta** lo que llega tras el hueco, y al vencer el **timer único** se retransmite **el más viejo no confirmado y todos los siguientes**. Simple pero **desperdicia `N−1` tramas por pérdida**.
8. **Selective Repeat**: buffer en el receptor, **reordenamiento**, **NAK** (o **SACK**), y se retransmite **solo** lo que falta. Exige `2N ≤ M`.
9. **TCP con SACK (`RFC 2018`) es Selective Repeat a nivel de bytes**; **TCP sin SACK es Go-Back-N**. Y **ARQ + FEC** se combinan hoy en todo sistema real.
10. Para **2026**: `RFC 2018` sigue vigente, `RFC 9293` (2022) es la especificación de TCP, el **fast retransmit** reemplazó al timer en la detección, y el **satelital/radio** usan **FEC** en lugar de ARQ puro.

✅ Autoevaluación

Respondé sin mirar. Las respuestas están en `<details>`.

1. ¿Por qué `2^r ≥ m + r + 1` y no `2^r ≥ m + 1`?

<details>
<summary>Respuesta</summary>
</details>

2. En el ejemplo `m = 4`, `r = 3`, dato `1010`: ¿qué palabra de código se envía? ¿Y si se invierte el bit 6?

<details>
<summary>Respuesta</summary>
</details>

3. ¿Por qué SEC no detecta dos errores?

<details>
<summary>Respuesta</summary>
</details>

4. Calculá la eficiencia de Stop-and-Wait para un enlace de 4 Mbps a 5000 km (`v = 2 × 10⁸ m/s`) con tramas de 500 bytes.

▶ **Respuesta**

5. En Selective Repeat, ¿por qué la ventana debe cumplir `2N ≤ M` ?

▶ **Respuesta**

6. ¿En qué se diferencia exactamente TCP de Go-Back-N y de Selective Repeat?

▶ **Respuesta**

7. ¿Cuántas tramas de datos transmite cada protocolo si se pierde la tercera de cinco, con `N = 4` ?

▶ **Respuesta**

8. ¿Por qué en un enlace de satélite conviene corregir y no retransmitir?

▶ **Respuesta**

🔗 Referencias

- `eirc0903` / `sirc0903` — **Pregunta 1 (8 pts en el enunciado, 10 en la pauta)**: el principio de funcionamiento de **Go-Back-N**. Pregunta 2 (10 pts): enrutamiento jerárquico, IGP con jerarquías, hot-potato.
- `eirc0703` / `sirc0703` — **Problema 1 (30 pts)**: protocolo de capa 2 punto a punto con ARQ **Selective Repeat** y **piggybacking**. Tipos de trama, ventana máxima (`2N ≤ M` con `M = 256` ⇒ **128**), y el protocolo en pseudocode.
- `eirc1103` / `sirc1103` — **Pregunta 4 (10 pts)**: el diagrama de TCP con y sin pérdida del primer segmento, que es Go-Back-N sobre TCP sin SACK.
- `eirc1007` / `sirc1007` — **Problema 1 (30 pts)**: protocolo *stop-and-wait* semifiable para video MPEG-2 (solo los I-frame son confiables).
- `eirc0912` / `sirc0912` — Pregunta 2 (10 pts): servicio confiable en transporte vs. enlace.
- `eirc0707` / `sirc0707` — Pregunta 2 (5 pts): mecanismos de detección y corrección de errores de la capa de enlace.
- **Práctico 5 de 2008** (`prac05_2008-p01.png`) — ejercicios 1 a 4: paridad, paridad 2D, Internet checksum y **CRC** (todos en la Clase 39; el **Ejercicio 4 NO es de Hamming**, es de CRC).
- **Tanenbaum & Wetherall**, *Redes de computadores* 5ª ed., **cap. 3 §3.3** (detección de errores: Hamming, CRC) y **§3.4–3.5** (protocolos de la capa de datos: ARQ, *piggybacking*), y **cap. 4** (capas inferiores)
- **Kurose & Ross**, *Computer Networking* 8ª ed., §5.3 (ARQ: Go-Back-N y Selective Repeat) y Figura 5.4 (el código de Hamming)
- **RFC 2018** — TCP Selective Acknowledgment (SACK), *Standards Track* · **RFC 9293** — TCP (2022, *Internet Standard*) · **RFC 6928** — Initial Window = 10 MSS

- **Clase anterior:** 39 — Detección de errores: paridad, checksum y CRC
- **Clase siguiente:** 41 — Acceso al medio: ALOHA, CSMA/CD y CSMA/CA